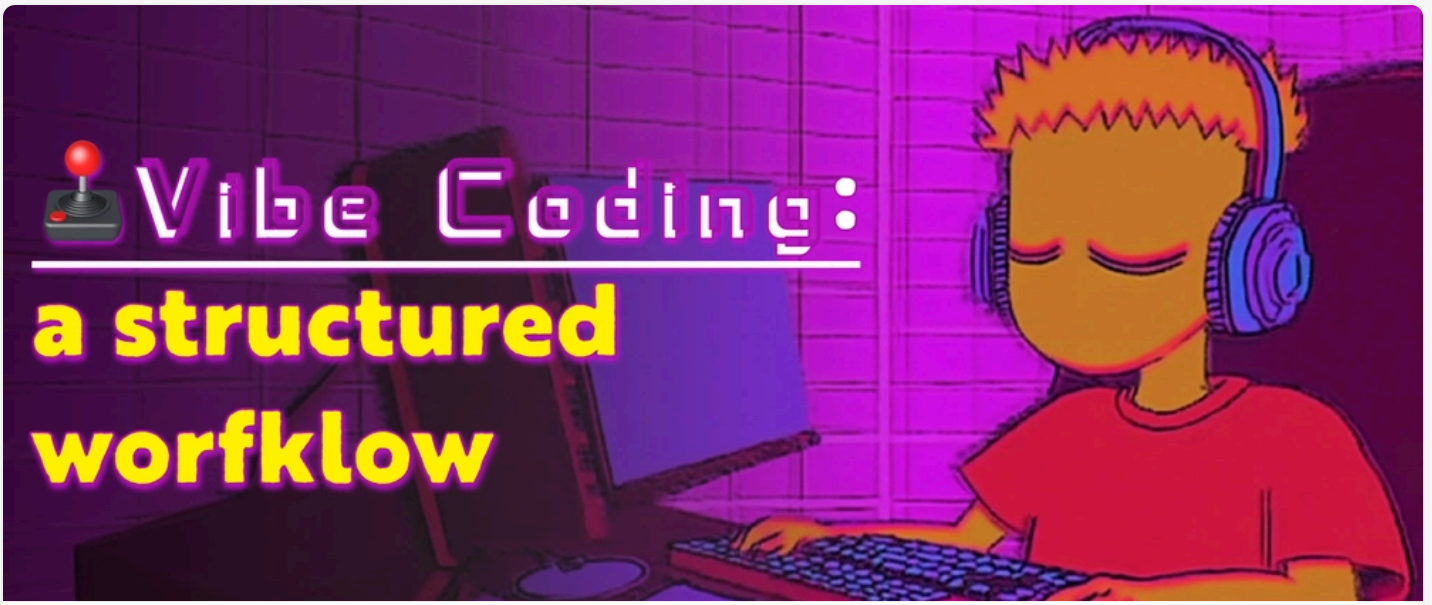


vincanger for Wasp
Posted on 16 Apr

A Structured Workflow for "Vibe Coding" Full-Stack Apps

#vibecoding #ai #fullstack #webdev

There's a lot of hype surrounding "vibe coding". You've probably seen the AI influencers making claims like how you can build SaaS apps in 15 minutes with just a few tools and prompts.

But, as you might have guessed, these example workflows are pretty flimsy.

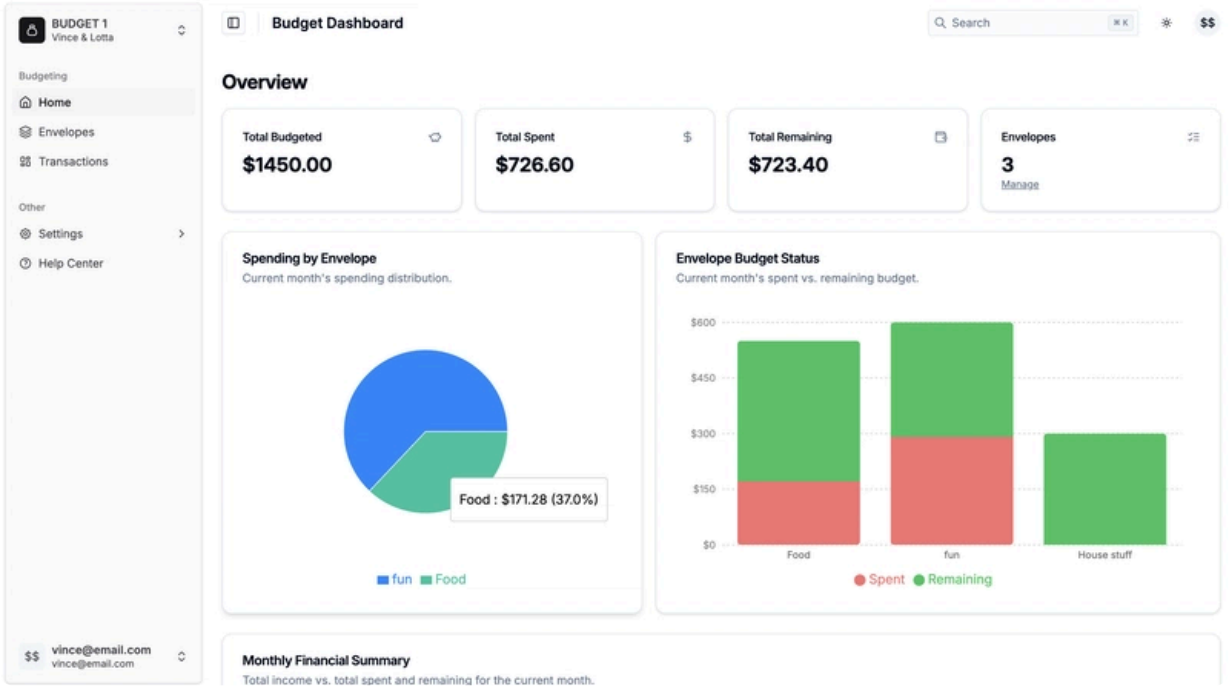
Yes, you can copy a landing page, or build a decent CRUD app, but you're not gonna be able to build a complex SaaS or internal tool with them.

But that doesn't mean there aren't workflows out there that can positively augment your development workflow. Anyone who's tinkered around with different AI-assisted techniques can tell you that there is some real magic in these tools.

That's why I spent a couple weeks researching the best techniques and workflow tips and put them to the test by building a full-featured, full-stack app with them.

template.

By the way, I came up with this workflow by testing and building a full-stack personal finance app in my spare time, tweaking and improving the process the entire time. Then, after landing on a good template and workflow, I rebuilt the app again and recorded it entirely, from start to deployments, in a ~3 hour long youtube video (above).



This article is a summary of the key approaches to implementing this workflow.

Step 1: Laying the Foundation

That's why you should give your AI helper a helping hand by starting with a solid foundation and leveraging the tools we have at our disposal.

In practical terms this means using stuff like:

1. UI Component Libraries
2. Boilerplate templates
3. Full-stack frameworks with batteries-included

Component libraries and templates are great ways to give the LLM a known foundation to build upon. It also takes the guess work out of styling and helps those styles be consistent as the app grows.

Using a full-stack framework with batteries-included, such as Wasp for JavaScript (React, Node.js, Prisma) or Laravel for PHP, takes the complexity out of piecing the different parts of the stack together. Since these frameworks are opinionated, they've chosen a set of tools that work well together, and the have the added benefit of doing a lot of work under-the-hood. In the end, the AI can focus on just the business logic of the app.

Take Wasp's main config file, for example (see below). All you or the LLM has to do is define your backend operations, and the framework takes care of managing the server setup and configuration for you. On top of that, this config file acts as a central "source of truth" the LLM can always reference to see how the app is defined as it builds new features.

```
app vibeCodeWasp {
  wasp: { version: "^0.16.3" },
  title: "Vibe Code Workflow",
  auth: {
    userEntity: User,
    methods: {
      email: {},
      google: {},
      github: {},
    },
  },
  client: {
    rootComponent: import Main from "@src/main",
    setupFn: import QuerySetup from "@src/config/querySetup",
  },
}
```

```
}

route EnvelopesRoute { path: "/envelopes", to: EnvelopesPage }
page EnvelopesPage {
  authRequired: true,
  component: import { EnvelopesPage } from
"@src/features/envelopes/EnvelopesPage.tsx"
}

query getEnvelopes {
  fn: import { getEnvelopes } from
"@src/features/envelopes/operations.ts",
  entities: [Envelope, BudgetProfile, UserBudgetProfile] // Need
BudgetProfile to check ownership
}

action createEnvelope {
  fn: import { createEnvelope } from
"@src/features/envelopes/operations.ts",
  entities: [Envelope, BudgetProfile, UserBudgetProfile] // Need
BudgetProfile to link
}

//...
```


Step 2: Getting the Most Out of Your AI Assistant

Once you've got a solid foundation to work with, you need create a comprehensive set of rules for your editor and LLM to follow.

To arrive at a solid set of rules you need to:

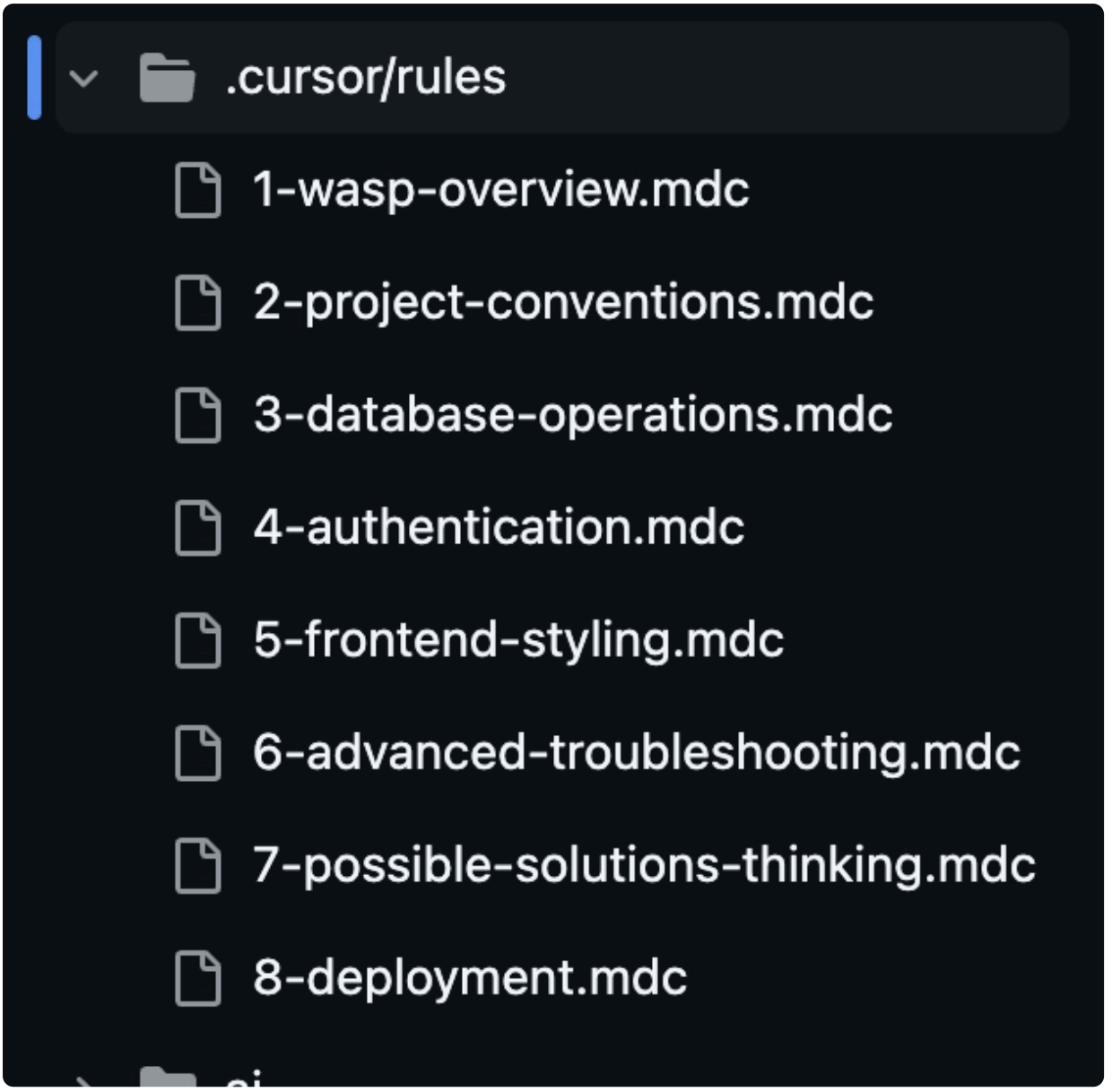
1. Start building something
2. Look out for times when the LLM (repeatedly) *doesn't meet your expectations* and define rules for them
3. Constantly ask the LLM to help you improve your workflow

Defining Rules

Different IDE's and coding tools have different naming conventions for the rules you define, but they all function more or less the same way (I used Cursor for this project so I'll be referring to Cursor's conventions here).

that align with your coding style, and project-specific rules (e.g. conventions, operations, auth).

The key here is to provide structured context for the LLM so that it doesn't have to rely on broader knowledge.



What does that mean exactly? It means telling the LLM about the current project and template you'll be building on, what conventions it should use, and how it should deal with common issues (e.g. the examples picture above, which are taken from the tutorial video's accompanying repo.

You can also add general strategies to rules files that you can manually reference in chat windows. For example, I often like telling the LLM to "think about 3 different strategies/approaches, pick the best one, and give your rationale for why you chose it." So I created a rule for it, `7-possible-solutions-thinking.mdc`, and I pass it in whenever I want to use it, saving myself from typing the same thing over and over.

Aside from this, I view the set of rules as a fluid object. As I worked on my apps, I started with a set of rules and iterated on them to get the kind of output I was looking for. This meant adding new rules to deal with common errors the LLM would introduce, or to overcome project-specific issues that didn't meet the general expectations of the LLM.

As I amended these rules, I would also take time to use the LLM as a source of feedback, asking it to critique my current workflow and find ways I could improve it.

This meant passing in my rules files into context, along with other documents like Plans and READMEs, and ask it to look for areas where we could improve them, using the past chat sessions as context as well.

A lot of time this just means asking the LLM something like:

> Can you review for breadth and clarity and think of a few ways it could be improved, if necessary. Remember, these documents are to be used as context for AI-assisted coding workflows.

Step 3: Defining the "What" and the "How" (PRD & Plan)

An extremely important step in all this is the initial prompts you use to guide the generation of the Product Requirement Doc (PRD) and the step-by-step actionable plan you create from it.

The PRD is basically just a detailed guideline for how the app should look and behave, and some guidelines for how it should be implemented.

After generating the PRD, we ask the LLM to generate a step-by-step actionable plan that will implement the app in phases using a modified **vertical slice method** suitable for LLM-assisted development.

The vertical slice implementation is important because it instructs the LLM to develop the app in full-stack "slices" -- from DB to UI -- in increasingly complexity. That might look like developing a super simple version of a full-stack feature in an early phase, and then adding more complexity to that feature in the later phases.

focused chunks

After the initial generation of each of these docs, I will often ask the LLM to review it's own work and look for possible ways to improve the documents based on the project structure and the fact that it will be used for assisted coding. Sometimes it finds seem interesting improvements, or at the very least it finds redundant information it can remove.

Here is an example prompt for generating the step-by-step plan (all example prompts used in the walkthrough video can be found in the accompanying repo):

> From this PRD, create an actionable, step-by-step plan using a modified vertical slice implmentation approach that's suitable for LLM-assisted coding. Before you create the plan, think about a few different plan styles that would be suitable for this project and the implmentation style before selecting the best one. Give your reasoning for why you think we should use this plan style. Remember that we will constantly refer to this plan to guide our coding implementation so it should be well structured, concise, and actionable, while still providing enough information to guide the LLM.

Finding this tutorial interesting?

Wasp team is working hard to create content like this, not to mention building a modern, open-source React/NodeJS framework.

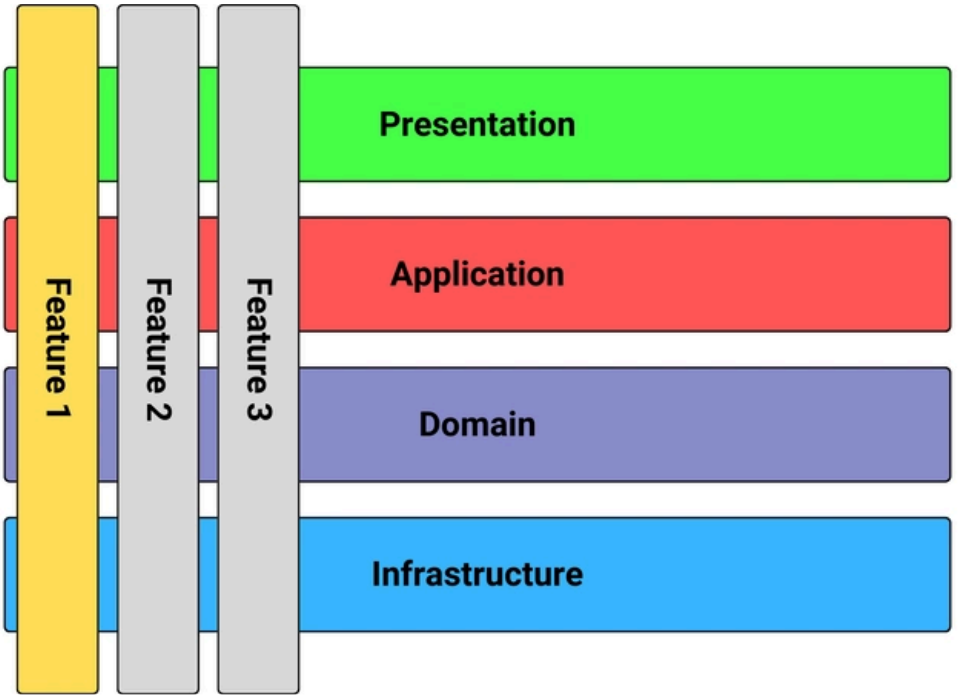
The easiest way to show your support is just to star Wasp repo! 🐝 But it would be greatly appreciated if you could take a look at the repository (for contributions, or to simply test the product). Click on the button below to give Wasp a star and show your support!

⭐ Thank You For Your Support 💪

Step 4: Building End-to-End - Vertical Slices in Action

As mentioned above, the vertical slice approach lends itself well to building with full-stack frameworks because of the heavy-lifting they can do for you and the LLM.

Rather than trying to define all your database models from the start, for example, this approach tackles the simplest form of a full-stack feature individually, and then builds upon them in later phases. This means, in an early phase, we might only define the database models needed for Authentication, then its related server-side functions, and the UI for it like Login forms and pages.



like:
-> Define necessary DB entities in `schema.prisma` for that feature only
-> Define operations in the `main.wasp` file
-> Write the server operations logic
-> Define pages/routes in the `main.wasp` file
-> src/features or src/components UI
-> Connect things via Wasp hooks and other library hooks and modules (react-router-dom, recharts, tanstack-table).

This gave me and the LLM a huge advantage in being able to build the app incrementally without getting too bogged down by the amount of complexity.

Once the basis for these features was working smoothly, we could improve the complexity of them, and add on other sub-features, with little to no issues!

The other advantage this had was that, if I realised there was a feature set I wanted to add on later that didn't already exist in the plan, I could ask the LLM to review the plan and find the best time/phase within it to implement it. Sometimes that time was then at the moment, and other times it gave great recommendations for deferring the new feature idea until later. If so, we'd update the plan accordingly.

Step 5: Closing the Loop - AI-Assisted Documentation

Documentation often gets pushed to the back burner. But in an AI-assisted workflow, keeping track of why things were built a certain way and how the current implementation works becomes even more crucial.

The AI doesn't inherently "remember" the context from three phases ago unless you provide it. So we get the LLM to provide it for itself :)

After completing a significant phase or feature slice defined in our Plan, I made it a habit to task the AI with documenting what we just built. I even created a rule file for this task to make it easier.

The process looked something like this:

- Gather the key files related to the implemented feature (e.g., relevant sections of main.wasp, schema.prisma, the operations.ts file, UI

feature.
- Reference the rule file with the Doc creation task
- Have it review the Doc for breadth and clarity

What's important is to have it focus on the core logic, how the different parts connect (DB -> Server -> Client), and any key decisions made, referencing the specific files where the implementation details can be found.

The AI would then generate a markdown file (or update an existing one) in the `ai/docs/` directory, and this is nice for two reasons:

1. For Humans: It created a clear, human-readable record of the feature for onboarding or future development.
2. For the AI: It built up a knowledge base within the project that could be fed back into the AI's context in later stages. This helped maintain consistency and reduced the chances of the AI forgetting previous decisions or implementations.

This "closing the loop" step turns documentation from a chore into a clean way of maintaining the workflow's effectiveness.

Conclusion: Believe the Hype... Just not All of It

So, can you "vibe code" a complex SaaS app in just a few hours? Well, kinda, but it will probably be a boring one.

But what you can do is leverage AI to significantly augment your development process, build faster, handle complexity more effectively, and maintain better structure in your full-stack projects.

The "Vibe Coding" workflow I landed on after weeks of testing boils down to these core principles:

- **Start Strong**: Use solid foundations like full-stack frameworks (Wasp) and UI libraries (Shadcn-admin) to reduce boilerplate and constrain the problem space for the AI.
- **Teach Your AI**: Create explicit, detailed rules (.cursor/rules/) to guide the AI on project conventions, specific technologies, and common pitfalls. Don't rely on its general knowledge alone.
- **Structure the Dialogue**: Use shared artifacts like a PRD and a step-by-step Plan (developed collaboratively with the AI) to align intent and break down work.

Use the AI to help document features as you build them, maintaining project knowledge for both human and AI collaborators.

- **Iterate and Refine**: Treat the rules, plan, and workflow itself as living documents, using the AI to help critique and improve the process.

Following this structured approach delivered really good results and I was able to implement features in record time. With this workflow I could really build complex apps 20-50x faster than I could before.

The fact that you also have a companion that has a huge knowledge set that helps you refine ideas and test assumptions is amazing as well

Although you can do a lot without ever touching code yourself, it still requires you, the developer, to guide, review, and understand the code. But it is a realistic, effective way to collaborate with AI assistants like Gemini 2.5 Pro in Cursor, moving beyond simple prompts to build full-features apps efficiently.

If you want to see this workflow in action from start to finish, check out the full ~3 hour YouTube walkthrough and template repo. And if you have any other tips I missed, please let me know in the comments :)

Top comments (37)

Subscribe

Add to the discussion

davide oliveri • 17 Apr

Thank you for the write up! I'm currently at the point where I experienced first hand the limitations beyond the hype and your guide gives a good starting point.

I haven't watched your video yet and I wonder if you mention anything about costs. How much did it cost to create the app from scratch with your approach?

3 likes Reply

I pay the $20/month flat fee for Cursor. Works great!

2 likes Reply

davide oliveri • 21 Apr

I understand. I thought that cursor users had to pay extra for api usage. Wasn't it your case for the use of Gemini 2.5?
Note: I am on the go and I can't review your article, but I recall you mentioning you've been using Gemini 2.5. I hope I'm not wrong.

1 like *Thread*

vincanger • 22 Apr

if you pay the flat fee you dont have to pay extra for using Gemini 2.5 pro. There are other models that cost extra per request though, just not this one :)

1 like Reply

Franjo Mindek • 16 Apr

Go Vinny! Go Vinny!

4 likes Reply

vincanger • 16 Apr

🏃

2 likes Reply

Mihovil Ilakovac • 16 Apr • Edited on 16 Apr

I'd love to see more visualizations on your way of thinking, it sounds like you went deep and got some really nice insights there! Also, what percentage of the code was written by you vs. AI?

4 likes Reply

I barely touched a line of code! So 98% ai written for sure. I'd suggest to do a code review after to gauge the quality.

6 likes Reply

Sasha • 16 Apr

Great writeup, and thanks for the video. For vibe coding, at least to me, it is better to see how people do it, over read about it.

4 likes Reply

vincanger • 22 Apr • Edited on 22 Apr

That was the idea behind it. Glad you find it useful. :)

1 like Reply

Matija Sosic • 16 Apr

This is exactly what I was hoping for. A full, detailed, step-by-step guide on how to build a medium-sized SaaS app with AI. All the quirks and errors included, together with deploying the app. Thanks, Vinny!

4 likes Reply

vincanger • 16 Apr

Thanks. Hope it's useful!

1 like Reply

Werewolf-XL • 18 Apr

I registered to this platform because of your article. This is a really well done workflow that I am gonna put to use in my own projects.

2 likes Reply

Awesome! Glad you found it useful.

1 like Reply

Roger St Hilaire • 23 Apr

Thank you for the write up I really appreciate the details and you sharing your process. It will definitely help me a a project I am building. Great work !!!

2 likes Reply

vincanger • 23 Apr

thanks. glad it could help :)

1 like Reply

Carlos Precioso • 16 Apr

Nice dive!

3 likes Reply

Paul Funnell • 18 Apr

This is great, I've been kicking these ideas around with GPT after Sam's Sunday letter on Lego coding but you are definitely a few steps ahead. I'm also working on self hosting the assistant for more control and cost efficiency when being scaled to a team.

1 like Reply

vincanger • 22 Apr

Nice. Good luck! :)

1 like Reply

You should remove the God's image .it is not appropriately used .
You are hurting hindu sentiments

2 likes Reply

Sourabh • 17 Apr

Yes , I was about to comment the same.
This is very inappropriate

1 like Reply

Milica Maksimovic • 17 Apr

Apologies, we changed the cover as soon as we realized the mistake!

3 likes Reply

Stanley Sathler • 18 Apr

Great post, thanks! 🚀

You suggest to document continuously - how exactly? Comments in the code? More cursor rules? PRDs?

1 like Reply

vincanger • 22 Apr

I usually have it generate Documentation for each phase of the plan after I've implemented it. It gets more beneficial as the project grows in size and complexity, because then you can pass previous examples to it as reference so it stays consistent. Of course, Wasp helps a lot here in that it takes care of a lot of boilerplate code for the LLM.

2 likes Reply

Thanks! Question is more about where exactly you store those docs?

1 like Reply

Juraj • 17 Apr

🔥 🔥

1 like Reply

vincanger • 22 Apr

😎

1 like Reply

Digvijay Shelar • 19 Apr

Great one vinny

1 like Reply

vincanger • 22 Apr

Thanks :)

1 like Reply

J

J J • 23 Apr • Edited on 23 Apr

A "Structured Worfklow"? Ironic, perhaps?
That said, thanks for the article.

1 like Reply

vincanger • 23 Apr

maybe a little

1 like Reply

Nice! I will use Laravel, React, Inertia stacks.

Reply

vincanger • 22 Apr

It will probably work for that stack as well since it's a batteries-included framework like Wasp is for javascript (react, nodejs, prisma). It's just that we unfortunately don't have a template and predefined rules for laravel.

3 likes Reply

Oluwawunmi Adesewa • 18 Apr

This is hard asf.

Reply

vincanger • 22 Apr

Tuff!

1 like Reply

E FLAT MAJOR • 18 Apr

I code a full app using DeepSeek , all I need to do is copy and paste it into python

1 like Reply

vincanger • 22 Apr

that can work too to a certain extent.

1 like Reply

View full discussion (37 comments)

Wasp

Follow

The fastest way to develop web apps with React & Node.js

Support Wasp by starring it on GitHub! ⭐

Star Wasp

More from Wasp

Wasp: a full-stack, "Laravel for JS" framework. Vibe-coding approved

Going from an Idea to MVP in Weeks: PromptPanda's Launch(es)

#webdev #saas #javascript #buildinpublic

From “You will fail” to 15,000 GitHub stars: The story of Wasp, a "Laravel for JS" full-stack framework

#webdev #javascript #ai #react